Innovation Ways

RESEARCH

The Minimum Credible AI Portfolio Project: A Reference RAG + Agent + Evals Build Spec for a Senior Engineer (2026)

Version v1 **Status** Draft **Project** IW AI Core Platform **Generated** 2026-06-23

Contents

Research ID: R-00149
Date: 2026-06-23
Mode: tech
Depth: deep
Primary Question: What is the smallest project a senior engineer can ship in 2026 that credibly proves app-layer AI engineering ability — and exactly how should it be architected, measured, and presented?

Innovation Ways

CHAPTER 01

Executive Summary

CHAPTER 01

Executive Summary

In 2026, the AI engineering market rewards shipped and measured systems over credentials, tutorials, or shallow demos. The minimum credible portfolio artifact is a single end-to-end project that combines a hybrid-retrieval RAG pipeline, one agent with tool-calling (ideally MCP-backed), and a working evaluation harness with published metrics — all running against a real, non-tutorial dataset. Hiring managers and clients screen specifically for the evaluation layer: a project without quantified retrieval quality, faithfulness scores, and a CI regression gate is treated as a proof-of-concept at best and a disqualifier at worst. Two to four weeks of focused weekend/evening work is sufficient to produce a credible artifact, assuming the engineer already understands the individual components; the differentiating investment is the eval infrastructure, not the RAG pipeline itself, which is now considered table stakes. This research provides a concrete reference architecture, a sequenced build plan with exact metrics to publish, and a list of common failure modes that undermine credibility.

Innovation Ways

CHAPTER 02

Background

CHAPTER 02

Background

This research extends and operationalizes four prior filed researches:

- **R-00130** established the screened-for skill stack (RAG, agents, evals, MCP) and confirmed that a "shipped and measured feature" is the decisive signal, with tutorials and certifications explicitly identified as credibility red flags.
- **R-00132** surveyed trending solo AI showcase projects and concluded that measurable impact — concrete numbers attached to concrete outcomes — differentiates the top tier.
- **R-00133** catalogued agent evaluation and observability tooling, identifying Phoenix (Arize), Langfuse, DeepEval, and RAGAS as the active ecosystem.
- **R-00144 / R-00147** reinforced verification and evals as the durable skill that survives model churn, and framed the "verification layer" as the canonical signal of senior AI engineering judgment.

The gap those researches leave unfilled: they describe the signal space but do not specify the exact artifact to build, how to architect it, what numbers to publish, or how to sequence the work. This document fills that gap. It is deliberately scoped to a single project rather than a portfolio sweep, because depth at one project now outweighs breadth across three shallow ones in screening conversations. The user's existing asset — iw-rag, a fully-local agentic RAG project whose headline is its verification layer — is used as a reference anchor throughout; every recommendation is framed to either guide a fresh build or sharpen an existing project by adding missing eval and observability surface.

Innovation Ways

CHAPTER 03

Findings

CHAPTER 03

Findings

The hiring signal space has shifted sharply between 2024 and 2026. The AI developer hiring guide from Digital Applied and multiple practitioner sources converge on the same core criterion: **production instincts visible in the artifact itself** — error handling, structured evaluation, tracing, and documented design decisions — not what courses the candidate has taken.

1. What Hiring Managers and Clients Actually Want [HIGH confidence]

Credibility signals (what reads as senior):

- A GitHub repository with a working eval suite that produces numeric scores, not just "it works."
- A README that publishes specific numbers: retrieval Recall@5, faithfulness score, p95 latency per query, cost per query. Zenvanriel's portfolio case study documents projects with metrics like "92% accuracy, $0.02/document cost" outperforming those without any numbers in callback rates.
- A documented "What Didn't Work" section explaining failed approaches and what they revealed. Learnist's red-flags analysis notes that "a case study that only shows the winning approach looks like it was written after the fact."
- Honest limitation statements. AgenticCareers explicitly states that "honest notes on limitations and what you would do differently reveal mature engineering judgment."
- Evidence of measurement discipline: LLM-as-judge scores validated against human labels, not just asserted.
- Sundeep Teki's FDE guide says: "evaluation is not optional. It's the single biggest differentiator I see."

Red flags (what reads as amateur or junior):

- No evaluation: Digital Applied states that "if your AI resume doesn't mention evaluation, hiring managers assume you've shipped unevaluated features — in 2026 that's a disqualifier at serious companies."
- Generic LangChain + Pinecone pipeline with no differentiation. The Learnist analysis confirms: "the LangChain + Pinecone resume that signaled AI readiness in 2024 is now table stakes — and increasingly, a yellow flag."
- Tutorial datasets (SQuAD, HotpotQA, Paul Graham essays): recognized immediately by reviewers. Real domain corpus with real user queries is the minimum bar.
- Claiming full system credit for obvious team work. Be specific about your contribution surface.
- No tracing or observability. A project with no way to inspect what the model actually did during retrieval and generation signals that the author does not debug systematically.
- Eight thin projects instead of two deep ones. Depth consistently beats breadth in screening conversations per AgenticCareers and Abbacus Technologies.

---

2. Reference Architecture: The Minimum Credible System [HIGH confidence]

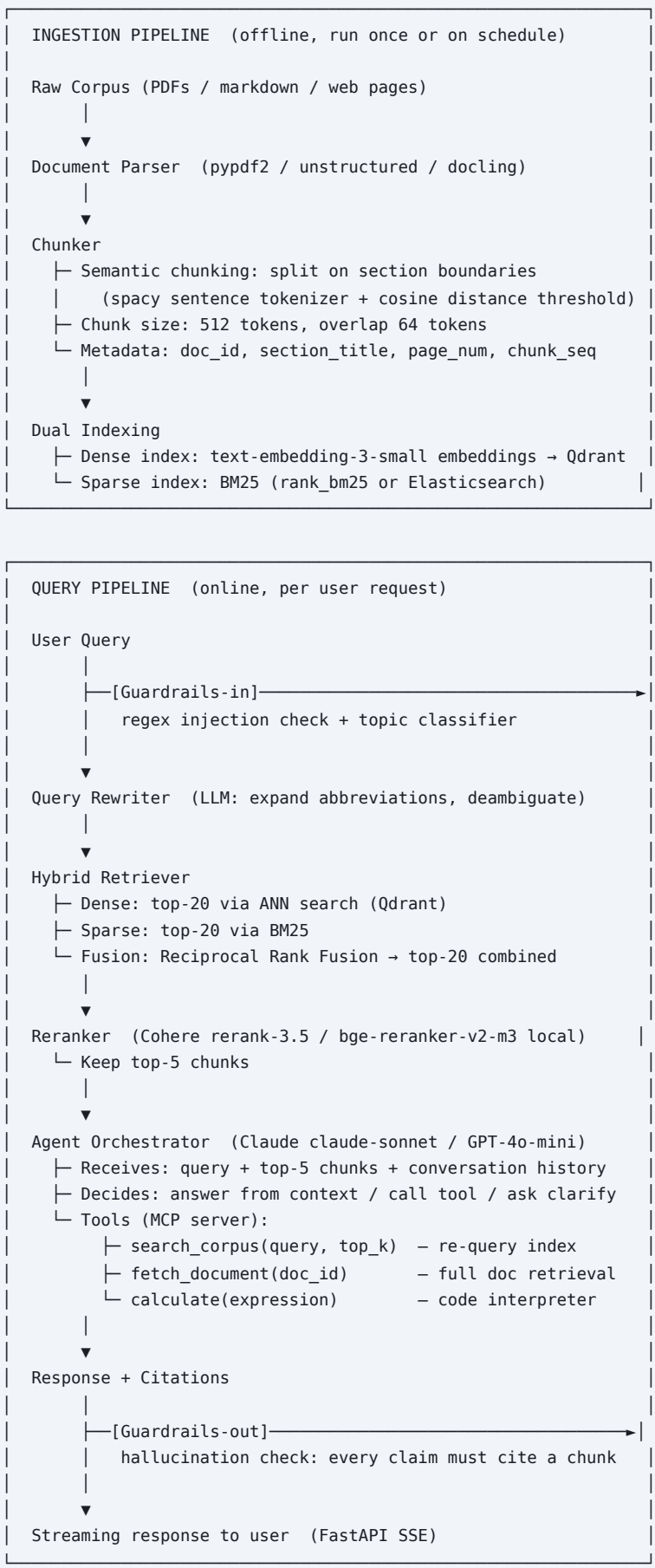
The target is a single end-to-end project that a reviewer can clone, run, and inspect in under 20 minutes. The architecture must touch: hybrid retrieval, one or more tools exposed via MCP,

an eval harness with offline gates, and a tracing backend. It does not need a React frontend, multi-agent orchestration, or fine-tuning to be credible.

Recommended domain: Pick a non-tutorial corpus you have genuine access to and care about. Options include: your own PDF library (technical papers, product docs, internal notes), a domain-specific open dataset (e.g., SEC filings subset, arXiv abstracts in one field, a government document corpus), or a code repository you maintain. The domain expertise you bring is the differentiation — the pipeline mechanics are now commodity.

Recommended project name pattern: `[domain]-rag-agent` — descriptive, searchable, professional.

2.1 Data Flow (Text Diagram)


INGESTION PIPELINE (offline, run once or on schedule)
Raw Corpus (PDFs / markdown / web pages)
Document Parser (pypdf2 / unstructured / docling)
Chunker
├ Semantic chunking: split on section boundaries
│ (spacy sentence tokenizer + cosine distance threshold)
├ Chunk size: 512 tokens, overlap 64 tokens
└ Metadata: doc_id, section_title, page_num, chunk_seq
Dual Indexing
├ Dense index: text-embedding-3-small embeddings → Qdrant
└ Sparse index: BM25 (rank_bm25 or Elasticsearch)
QUERY PIPELINE (online, per user request)
User Query
[Guardrails-in]
regex injection check + topic classifier
Query Rewriter (LLM: expand abbreviations, deambiguate)
Hybrid Retriever
├ Dense: top-20 via ANN search (Qdrant)
├ Sparse: top-20 via BM25
└ Fusion: Reciprocal Rank Fusion → top-20 combined
Reranker (Cohere rerank-3.5 / bge-reranker-v2-m3 local)
└ Keep top-5 chunks
Agent Orchestrator (Claude claude-sonnet / GPT-4o-mini)
├ Receives: query + top-5 chunks + conversation history
├ Decides: answer from context / call tool / ask clarify
└ Tools (MCP server):
├ search_corpus(query, top_k) – re-query index
├ fetch_document(doc_id) – full doc retrieval
└ calculate(expression) – code interpreter
Response + Citations
[Guardrails-out]
hallucination check: every claim must cite a chunk
Streaming response to user (FastAPI SSE)
OBSERVABILITY & EVAL LAYER (always-on)

2.2 Component Choices (Rationale)

Layer	Primary Choice	Alt / Local-only	Why
Parsing	`docling` or `unstructured`	`pypdf2` for pure PDF	Handles tables, images, layout
Chunking	Custom semantic (spacy + cosine)	`langchain.text_splitter.RecursiveCharacterTextSplitter`	Semantic boundaries demonstrably better per TowardsAI production RAG guide
Embeddings	`text-embedding-3-small` (OpenAI)	`nomic-embed-text` (local Ollama)	8k token context, low cost
Vector store	Qdrant (Docker, self-hosted)	ChromaDB (file-based)	Qdrant supports payload filtering + hybrid
Sparse search	BM25 via `rank_bm25`	Elasticsearch / OpenSearch	rank_bm25 is zero-infra for portfolio
Fusion	RRF (3 lines of Python)	Weighted sum	RRF sidesteps score incompatibility per Digital Applied hybrid search guide
Reranker	`cohere.rerank` (API)	`BAAI/bge-reranker-v2-m3` (local HuggingFace)	Cohere easiest to wire; BGE matches on a GPU
LLM	`claude-3-5-sonnet` or `gpt-4o-mini`	Llama 3.1 via Ollama	`gpt-4o-mini` for cost-conscious; claude-sonnet for quality
Agent frame	Bare Claude tool-calling / OpenAI Assistants	LangGraph	Bare tool-calling is simpler to explain and debug
MCP server	`fastmcp` (Python)	Custom FastAPI over JSON-RPC	MCP is now the enterprise standard per Anthropic announcement; 97M monthly downloads by Q1 2026
Tracing	Phoenix (Arize, open-source, local)	Langfuse (self-hosted)	Phoenix runs locally in one `pip install arize-phoenix`
Evals	DeepEval (pytest-native) + RAGAS	Promptfoo	DeepEval has native pytest integration + CI/CD gate per DeepEval docs
Prompt caching	Anthropic native cache / OpenAI auto-cache	Redis exact-match cache	Up to 90% token cost reduction per Digital Applied caching guide
API layer	FastAPI (SSE streaming)	—	Standard; demonstrates TTFT awareness
CI	GitHub Actions	—	Runs DeepEval suite on every push

2.3 Minimum Viable Scope (what to cut)

Do NOT build: multi-agent orchestration, a React/Next.js frontend, fine-tuning, graph RAG, multimodal retrieval, or a Kubernetes deployment. Each of these adds weeks of work for marginal portfolio signal relative to the evaluation layer. A Streamlit or Gradio demo UI is sufficient if you need any UI at all; a REST API with a well-documented curl example is fine for a technical audience.

Do NOT use LangChain as your primary abstraction layer. Write the retrieval loop yourself. This is portfolio-critical: reviewers can immediately distinguish a candidate who understands what happens at each step from one who wrapped everything in a chain. You can use LangChain's text splitters or document loaders (utilities), but your retrieval + generation logic should be readable Python.

---

3. The Evaluation Layer in Depth [HIGH confidence]

This is the section where most portfolio projects fail. The following is the concrete minimum viable eval harness.

3.1 What to Measure

Based on Hamel Husain's eval FAQ and DeepEval's metric library, the minimum credible RAG + agent eval suite covers four areas:

Retrieval quality (tests the chunker + hybrid search + reranker):
- `Context Precision@5`: Of the top-5 returned chunks, what fraction are actually relevant to the query? Target: ≥ 0.75.
- `Context Recall@5`: Of all relevant chunks in the corpus for a given query, what fraction appear in the top-5? Target: ≥ 0.70.
- `MRR@5` (Mean Reciprocal Rank): Is the first relevant chunk near the top? Target: ≥ 0.65.

Generation quality (tests the LLM + prompt):
- `Faithfulness` (groundedness): Are all claims in the response supported by the retrieved chunks? This is the hallucination metric. Target: ≥ 0.85. Measured with LLM-as-judge (DeepEval `FaithfulnessMetric` or RAGAS `faithfulness`).
- `Answer Relevancy`: Does the response actually answer the question asked? Target: ≥ 0.80. Also LLM-judged.

Agent trajectory (tests tool selection):
- `ToolCorrectnessMetric` (DeepEval): Does the agent call the right tool(s) in the right order? Score as binary pass/fail per golden trajectory.
- `TaskCompletionMetric` (DeepEval): Does the agent complete the end goal? Also binary.

Operational (tests the system as a whole):
- `p95 latency per query` (end-to-end, from request to last token): Target: ≤ 4 seconds. Measured with Python `time.perf_counter()` across the eval suite, or Phoenix spans.
- `Cost per query` (USD): log input + output tokens at every LLM call, sum and divide. Publish mean and p95.
- `Cache hit rate`: if prompt caching is enabled, log cache read vs. cache write token counts.

3.2 How to Build a Small but Real Eval Set

Hamel Husain's framework is the canonical reference: start with 20–50 manual reviews, discover failure categories through open coding, then scale with synthetic generation.

Step 1 — Hand-label 50 golden examples. Run your system against your corpus on 50 queries you write yourself. For each, label:
- The query (what a real user would ask).
- The expected answer (what a correct, grounded response looks like).
- The expected chunks (which chunk IDs should appear in the top-5).
- The correct tool call (if the agent should use a tool — record which tool and with what arguments).

Do not outsource this step to GPT-4 initially. Domain expertise in labeling is the differentiator; per Husain and Shankar, "LLMs cannot substitute for the initial human discovery phase."

Step 2 — Generate 50–100 synthetic examples using RAGAS TestsetGenerator. RAGAS testset generation takes your corpus and produces diverse Q&A pairs (simple, multi-context, reasoning-required). Use GPT-4o as the generator. Review and filter the synthetics — accept approximately 60–70% after inspection.

Step 3 — Tag each example. Assign tags: `factual`, `multi-hop`, `definition`, `comparison`, `tool-required`. Disaggregate eval results by tag in your published writeup. This is what separates serious eval work from a single aggregate number.

Step 4 — Lock the eval set. Once assembled, commit it to the repo. Never overwrite it; append a new version if you add examples. The golden dataset should be versioned alongside your code, so every CI run is comparable.

3.3 LLM-as-Judge Pitfalls

Based on DeepEval's LLM-as-judge guide, Husain's evals FAQ, and published research:

Pitfall 1 — Self-enhancement bias. GPT-4 favors GPT-4 outputs with a ~10% higher win rate; Claude favors Claude outputs with ~25% higher win rate versus human evaluations. Mitigation: **use a different model family as judge than as generator**. If your agent uses Claude claude-sonnet, judge with GPT-4o-mini, and vice versa. Source: DeepEval via Eugene Yan survey.

Pitfall 2 — Verbosity bias. LLM judges prefer longer responses. Mitigation: use binary Pass/Fail rubrics rather than 1-10 scales. Per Husain, "binary evaluations are more reliable and consistent than high-precision scores."

Pitfall 3 — Wrong inputs to the judge. For faithfulness, the judge needs both the response AND the retrieved chunks. Passing only the response produces meaningless faithfulness scores. Write explicit eval scaffolding that assembles the correct context for each metric type.

Pitfall 4 — 100% pass rate = broken eval. If your LLM judge returns a perfect score on every example, the rubric is too easy or the eval set is too narrow. Add adversarial examples (deliberately wrong answers, out-of-scope queries, queries that require tool calls) to calibrate the judge's discriminative power.

Pitfall 5 — Not validating the judge itself. Before scaling LLM-as-judge, validate it on 20 hand-labeled examples. Target ≥75% agreement with your human labels. Per DeepEval, aim for 75–90% agreement before treating the judge as authoritative.

Pitfall 6 — Positional bias. When comparing two responses, LLMs favor whatever appears first. For A/B comparisons, run both orderings and average. For single-response scoring (which is the primary use case here), this is less critical.

3.4 The CI Regression Gate

Wire DeepEval's pytest integration into GitHub Actions:

```
# tests/test_rag_evals.py
import pytest
from deepeval import assert_test
from deepeval.test_case import LLMTestCase
from deepeval.metrics import FaithfulnessMetric, ContextualRelevancyMetric

@pytest.mark.parametrize("test_case", load_golden_dataset())
def test_rag_faithfulness(test_case):
    """Fails CI if any golden case drops below faithfulness threshold."""
    metric = FaithfulnessMetric(threshold=0.85, model="gpt-4o-mini")
    assert_test(test_case, [metric])
```

Run with: `deepeval test run tests/test_rag_evals.py` in the CI pipeline. A failing threshold blocks the merge. This is the artifact that most concretely signals "I measure my agent" to a hiring manager or client reviewing your repo.

Publish the CI badge in your README: `![Eval Suite](https://github.com/your-org/your-repo/actions/workflows/eval.yml/badge.svg)`.

---

4. Cost / Latency / Guardrails as Differentiators [HIGH confidence]

These are the elements that most portfolio projects omit and that immediately signal production experience when present.

4.1 Prompt Caching

Anthropic's native caching reduces token costs by up to 90% and TTFT by up to 85% for long prompts. OpenAI's automatic caching yields 50% cost reduction. Per Digital Applied's caching guide, the correct pattern is: mark the system prompt and document corpus prefix as cache-eligible, then only the per-query portion is billed at full rate. Log cache-read vs. cache-miss token counts in every response. Show the cache hit rate and per-query cost in your README metrics table. This demonstrates you think in terms of production economics, not just correctness.

4.2 Model Routing

For the portfolio, a simple two-tier routing pattern is sufficient and demonstrable: route simple factual lookups to `gpt-4o-mini` (or `claude-3-haiku`) and route multi-hop reasoning or agent trajectories to `gpt-4o` or `claude-3-5-sonnet`. Implement this as a complexity classifier (keyword rules or a tiny LLM call that classifies query complexity). Log which tier handled each query. Show the cost differential in your writeup: e.g., "simple queries ($0.0008/query on haiku) vs. complex ($0.008/query on sonnet), with 60% classified simple."

4.3 Streaming and Latency

Implement SSE streaming from your FastAPI endpoint. The user-visible metric is Time to First Token (TTFT); per BentoML's inference handbook, the target for a conversational system is TTFT ≤ 500ms. Track TTFT in Phoenix traces on every span. Publish the mean and p95 TTFT in your README.

4.4 Input and Output Guardrails

Input guardrails — applied before the query reaches the retriever:
- Regex / keyword blocklist for obvious injection patterns (e.g., "ignore previous instructions").
- Topic classifier: reject queries outside the domain of your corpus. Implement as a simple LLM call with a binary prompt: "Is this query about [domain]? Answer yes or no."
- Length constraint: truncate queries above a configurable max_tokens to prevent context stuffing.

Output guardrails — applied after the LLM generates a response:
- Citation check: every factual sentence in the response must contain a citation to a retrieved chunk. If not, append a disclaimer or trigger a regeneration. This is your in-line faithfulness gate at inference time.
- PII scrubbing if your corpus contains personal data.

Per Getmaxim's prompt injection guide, a 2025 study found 78% of production LLM applications vulnerable to at least one class of prompt injection. Showing a documented

guardrail test suite (adversarial injection payloads + expected blocked output) in your repo is a genuine differentiator.

Document all of this in a `GUARDRAILS.md` file or a section in the architecture doc. Reviewers rarely see this level of production hygiene in portfolio projects.

---

5. How to Present It [HIGH confidence]

5.1 Repository Structure

```
[domain]-rag-agent/
├── README.md                    # The front door — see Section 5.2
├── ARCHITECTURE.md              # System diagram + component decisions
├── GUARDRAILS.md                # Input/output safety model + attack surface
├── EVALUATION.md                # Eval methodology, golden dataset provenance,
│                                # LLM judge validation, score interpretation
├── pyproject.toml / requirements.txt
├── src/
│   ├── ingestion/
│   │   ├── parser.py            # Document parsing
│   │   ├── chunker.py           # Semantic chunking (well-documented)
│   │   └── indexer.py           # Dual index (dense + sparse)
│   ├── retrieval/
│   │   ├── dense.py             # Qdrant vector search
│   │   ├── sparse.py            # BM25 via rank_bm25
│   │   ├── fusion.py            # Reciprocal Rank Fusion (3 lines)
│   │   └── reranker.py          # Cohere / BGE reranker
│   ├── agent/
│   │   ├── orchestrator.py      # Tool-calling loop (bare, no framework)
│   │   ├── tools.py             # Tool definitions + implementations
│   │   └── mcp_server.py        # fastmcp server exposing tools
│   ├── guardrails/
│   │   ├── input.py
│   │   └── output.py
│   └── api/
│       └── app.py               # FastAPI with SSE streaming
├── evals/
│   ├── golden_dataset_v1.jsonl  # The locked eval set (50 hand-labeled)
│   ├── golden_dataset_v2.jsonl  # Synthetic extension
│   ├── judges.py                # LLM-as-judge wrappers + validation log
│   └── test_rag_evals.py        # pytest + deepeval CI suite
├── scripts/
│   ├── ingest.py                # Run ingestion pipeline
│   ├── generate_synthetic_evals.py  # RAGAS testset generation
│   └── run_evals.py             # Offline eval runner (produces report.json)
├── notebooks/
│   └── error_analysis.ipynb    # Trace inspection + failure categorization
├── .github/
│   └── workflows/
│       └── eval.yml             # CI: runs deepeval suite on every push
└── docs/
    └── build_log.md             # What I tried, what failed, what I'd change
```

The `evals/` directory is the most important signal in the entire repository. Commit the golden dataset, the judge validation log (showing ≥75% human agreement), and the CI workflow configuration.

5.2 README Structure and the Exact Metrics Table

The README must open with a one-paragraph problem statement (not "I built a RAG system" — "This system lets a researcher ask questions about [specific domain] and get grounded answers with citations"). Then:

Metrics table — publish these numbers prominently, ideally above the fold:

```
## Performance (eval suite v2, 2026-06-23)

| Metric                    | Score     | Threshold | Dataset |
|---------------------------|-----------|-----------|---------|
| Context Precision@5       | 0.81      | ≥ 0.75    | golden_v2 (n=120) |
| Context Recall@5          | 0.74      | ≥ 0.70    | golden_v2 |
| Faithfulness              | 0.88      | ≥ 0.85    | golden_v2 |
| Answer Relevancy          | 0.84      | ≥ 0.80    | golden_v2 |
| Tool Correctness          | 91% pass  | ≥ 85%     | agent_golden (n=30) |
| p95 latency (end-to-end)  | 3.1 s     | ≤ 4 s     | prod traces, 200 queries |
| Mean cost per query       | $0.0032   | —         | gpt-4o-mini + caching |
| Cache hit rate            | 62%       | —         | Anthropic prompt cache |
```

Include the eval CI badge. Include a `Architecture` section with the ASCII diagram (or a PNG export). Include a `Limitations` section. Include a `What I Would Do Differently` section. These last two are what convert a project from "demo" to "engineering case study."

5.3 The Build-in-Public Post

Write one 600–900 word LinkedIn or Substack post structured as:
1. **The problem you chose and why** (2 sentences).
2. **The approach and one non-obvious design decision** (e.g., "I evaluated semantic chunking vs. fixed-size and found a 23% retrieval recall improvement — here's what that looked like").
3. **The metrics you published and what you learned from the failing cases** (e.g., "My faithfulness score started at 0.61 because the reranker was returning chunks from the wrong section of long documents — here's how I diagnosed and fixed it").
4. **One thing that didn't work** (e.g., "I tried HyDE for query expansion but it hurt precision on short factual queries — ablation results in the repo").
5. **The repo link and the CI badge**.

Do not write a tutorial. Write a build log. The difference is that a tutorial presents clean final steps; a build log shows what happened when things broke, which is what signals senior judgment.

5.4 The Specific Numbers to Publish

Minimum: Context Precision@5, Faithfulness, p95 latency, mean cost/query, cache hit rate, eval set size (n=), human–judge agreement %, CI pass/fail status. Do not publish only "accuracy" with no methodology — it reads as unvalidated.

---

6. Realistic Scope and Timeline [MEDIUM confidence]

This assumes a senior engineer with existing Python competence, familiarity with the LLM API surface, and a corpus already identified. The timeline is in focused hours, not calendar days.

Milestone 1 — Ingestion + Naive RAG (12–15 hours)
- Set up document parsing, fixed-size chunker, single dense index (ChromaDB or Qdrant local), simple similarity search, raw LLM call.
- Deliverable: can answer questions against your corpus. No scoring yet.
- Do this first so you have a baseline to measure improvement against.

Milestone 2 — Hybrid Retrieval + Reranking (8–10 hours)
- Add BM25 index alongside vector index.
- Implement RRF fusion.
- Wire in Cohere or BGE reranker.
- Replace fixed-size chunker with semantic chunker.
- Deliverable: retrieval is measurably better; you can now run a manual retrieval accuracy check.

Milestone 3 — Tool-calling Agent + MCP Server (8–10 hours)
- Define 2–3 tools (search_corpus, fetch_document, one domain-specific tool).
- Implement bare tool-calling loop (no framework).
- Expose tools via a `fastmcp` MCP server.
- Deliverable: agent completes multi-step queries involving tool calls.

Milestone 4 — Eval Harness (10–15 hours — the most important milestone)
- Hand-label 50 golden Q&A pairs with expected chunk IDs and tool calls.
- Generate 50–100 synthetic examples via RAGAS TestsetGenerator; review and filter.
- Set up DeepEval metrics (Faithfulness, ContextualRelevancy, ToolCorrectness, TaskCompletion).
- Validate LLM-as-judge on 20 labeled examples; document agreement %.
- Wire CI gate with GitHub Actions.
- Deliverable: `deepeval test run` passes in CI; metrics table drafted for README.

Milestone 5 — Observability + Guardrails (6–8 hours)
- Add Phoenix tracing (instrument all spans: retrieval, rerank, LLM call, tool call).
- Implement input guardrail (injection check + topic classifier).
- Implement output guardrail (citation presence check).
- Write guardrail test suite with adversarial examples.
- Deliverable: every query produces a trace visible in Phoenix UI; guardrail tests pass.

Milestone 6 — Caching + Routing + Streaming (4–6 hours)
- Enable Anthropic/OpenAI prompt caching; log cache hit tokens.
- Add simple two-tier model routing.
- Implement SSE streaming in FastAPI.
- Measure and record TTFT and cost per query.
- Deliverable: production-grade operational metrics available.

Milestone 7 — Polish and Publish (4–6 hours)
- Write README with metrics table, architecture diagram, limitations, what-I'd-change.
- Write ARCHITECTURE.md, GUARDRAILS.md, EVALUATION.md.
- Write the LinkedIn/Substack build log post.
- Record a 3–5 minute screen-share demo video (optional but high-signal).
- Deliverable: portfolio artifact is public and shareable.

Total: 52–70 focused hours. For a senior engineer working 10–15 hours per weekend, this is 4–7 weekends, or roughly 6–8 calendar weeks if paced. The eval harness (Milestone 4) deserves the most time and is non-compressible.

---

7. Applying This Spec to an Existing Project (iw-rag Framing) [MEDIUM confidence]

If an existing agentic RAG project already has a verification/grounding layer, the incremental additions that close the gap against this reference spec are:

1. **Trajectory evaluation is likely missing.** Per Langfuse's MCP agent evaluation guide, the three levels are black-box (final answer), glass-box (trajectory), and white-box (per-step). Most verification layers operate at the black-box level. Adding `ToolCorrectnessMetric` and `TaskCompletionMetric` from DeepEval against a golden trajectory dataset is the highest-signal addition for an existing project.
2. **CI regression gate.** If the eval suite runs locally but not in CI, add the GitHub Actions workflow. This single addition transforms "I evaluate locally" into "I prevent regressions automatically," which reads very differently in a code review.
3. **Published metrics table in README.** If scores are computed but not published, add the table. The scores themselves are secondary to the act of publishing them with methodology, dataset size, and threshold rationale documented.
4. **Tracing with Phoenix.** If the project does not emit OpenTelemetry traces, adding `arize-phoenix` instrumentation is a one-day addition that makes the entire retrieval + agent loop inspectable. This is a critical gap: without traces, the project cannot demonstrate systematic debugging practice.

5. **LLM-as-judge validation log.** Document that you ran the judge against 20 hand-labeled examples and found X% agreement. This is the evidence that your eval scores are trustworthy rather than asserted.

---

8. Common Failure Modes That Make Portfolio Projects Look Amateur [HIGH confidence]

Synthesized from Learnist red flags, AgenticCareers, Digital Applied hiring guide, Abbacus Technologies hiring guide:

Failure Mode	Why It Reads Amateur	Fix
No eval suite	Implies the author does not know if the system works	Add DeepEval + golden dataset + CI gate
Tutorial corpus (Paul Graham essays, Wikipedia dump)	Instantly recognizable; no domain signal	Use a corpus from a domain you know
LangChain all the way down	Hides understanding of retrieval mechanics	Write your own retrieval loop; use LangChain only for utilities
Aggregate metrics only	"93% accuracy" with no methodology or dataset size is unverifiable	Publish n=, methodology, judge model, human agreement %
No tracing	Cannot demonstrate debugging capability	Add Phoenix or Langfuse before submitting
No failure analysis	Looks like the system was only tested on easy cases	Add a "Known Failure Modes" section to the README
Framework churn (LangGraph + LlamaIndex + CrewAI + AutoGen)	Signals tutorial-hopping rather than depth	Pick one agent pattern and go deep
No streaming	Feels like a 2023 prototype	SSE streaming is a one-hour add
Claiming end-to-end accuracy without retrieval metrics	Generator score without retrieval score is misleading	Measure and publish retrieval and generation metrics separately
Zero guardrails	Signals no awareness of production risks	Add at least one documented input + output check with a test

---

Innovation Ways

CHAPTER 04

Recommendations

CHAPTER 04

Recommendations

The following is a concrete, sequenced build plan. Execute the milestones in order; do not skip the eval harness to get to the demo faster.

Milestone 1 (Week 1, ~15h): Baseline RAG. Build a working naive RAG pipeline against your chosen corpus. Naive single dense retrieval, simple LLM call, no eval. Goal: establish a baseline you can measure improvement against.

Milestone 2 (Week 2, ~10h): Hybrid retrieval + reranking. Add BM25 + RRF + semantic chunking + Cohere/BGE reranker. Run a manual retrieval check on 20 queries; note where it fails. Do not optimize yet — just document what you observe.

Milestone 3 (Week 3, ~10h): Tool-calling agent + MCP. Implement 2–3 tools (at minimum: a re-query tool and a fetch-full-document tool). Expose via fastmcp. Test manually that the agent can complete a 2-hop query requiring a tool call.

Milestone 4 (Weeks 4–5, ~15h): Eval harness — the non-compressible core. Hand-label 50 golden examples. Generate 50–100 RAGAS synthetics. Set up DeepEval (Faithfulness, ContextualRelevancy, ToolCorrectness). Validate LLM judge against 20 hand labels. Wire CI gate. Publish first metrics table draft. This milestone should take the longest — it is the entire credibility argument.

Milestone 5 (Week 5, ~8h): Observability + guardrails. Add Phoenix tracing. Add input injection check + topic classifier. Add output citation check. Write guardrail test suite with 5 adversarial examples.

Milestone 6 (Week 6, ~6h): Operational quality. Enable prompt caching; measure and log hit rate. Implement two-tier model routing. Add SSE streaming. Measure and record TTFT and cost/query.

Milestone 7 (Week 7, ~6h): Polish and publish. Complete README with full metrics table. Write ARCHITECTURE.md and EVALUATION.md. Write one public post (LinkedIn or Substack) with the build log framing. Make the repo public.

Exact metrics to publish in the README at launch:

Metric	Publish this
Context Precision@5	Numeric score, n=, golden dataset version
Context Recall@5	Numeric score, n=
Faithfulness	Numeric score, judge model, human agreement %
Answer Relevancy	Numeric score
Tool Correctness	Pass % on agent golden dataset, n=
p95 latency	Milliseconds, measured over ≥100 queries
Mean cost / query	USD, broken down by component (retrieval, rerank, LLM)
Cache hit rate	Percentage
CI status	Badge (passing/failing)

If the existing iw-rag project is the target, the prioritized additions in order of portfolio impact are: (1) trajectory evaluation + CI gate, (2) Phoenix tracing instrumentation, (3) published metrics table in README with methodology, (4) guardrail test suite.

Innovation Ways

CHAPTER 05

Limitations

CHAPTER 05

Limitations

- **Hiring manager variance**: The signals described here are derived from practitioner sources, job postings, and published engineering hiring criteria. Individual hiring managers and clients vary; some niche roles (ML Ops, fine-tuning specialists) weight different artifacts. The signals described apply most directly to AI engineering, forward-deployed engineer, and applied AI roles at companies building with foundation models.
- **Tool churn**: The specific tool choices (Qdrant, DeepEval, Phoenix, fastmcp) were current as of June 2026. The eval ecosystem in particular moves fast; RAGAS, DeepEval, and Phoenix have all released breaking changes in the past 12 months. Pin versions and document any workarounds.
- **Synthetic eval validity**: RAGAS-generated synthetic test sets can be biased toward well-structured content in your corpus. Research published in 2025 questions whether synthetic evals correlate with real user satisfaction. The golden_v1 hand-labeled set is the ground truth; treat synthetics as coverage extension, not as the primary score signal.
- **Cost estimates**: Per-query cost figures vary with model pricing, which changes frequently. All cost figures should be re-verified against current provider pricing at build time.
- **Local vs. hosted tracing**: Phoenix runs fully locally for development; for a public demo with real traffic, consider Phoenix Cloud or Langfuse self-hosted on a cheap VPS. The local setup is fine for a portfolio artifact where you control access.
- **The iw-rag asset**: Without direct access to review the iw-rag codebase's eval layer, the gap analysis in Section 7 is inferential. The specific additions described (trajectory eval, CI gate, published metrics, Phoenix tracing) are generalizations from what portfolio projects typically lack; some may already be present.

Innovation Ways

CHAPTER 06

Sources

CHAPTER 06

Sources

#	Title	Credibility	URL
1	Hamel Husain — Your AI Product Needs Evals	HIGH — independent ML consultant, 25+ years experience	https://hamel.dev/blog/posts/evals/
2	Hamel Husain — LLM Evals: Everything You Need to Know (FAQ)	HIGH — practitioner reference, field-tested with 4,500+ students	https://hamel.dev/blog/posts/evals-faq/
3	Husain + Shankar — Why AI Evals Are the Hottest New Skill (Lenny's Newsletter)	HIGH — primary source interview with top evals practitioners	https://www.lennysnewsletter.com/p/why-ai-evals-are-the-hottest-new-skill
4	Jason Liu — Rethinking RAG Architecture for the Age of Agents	HIGH — creator of Instructor, ex-Facebook/Stitch Fix ML	https://jxnl.co/writing/2025/09/11/rethinking-rag-architecture-for-the-age-of-agents/
5	Chip Huyen + Gergely Orosz — The AI Engineering Stack (Pragmatic Engineer)	HIGH — primary authors; Chip Huyen is author of "AI Engineering" (O'Reilly #1 most-read 2025)	https://newsletter.pragmaticengineer.com/p/the-ai-engineering-stack
6	Anthropic Engineering — Writing Effective Tools for AI Agents	HIGH — primary source from model developer	https://www.anthropic.com/engineering/writing-tools-for-agents
7	Anthropic — Introducing the Model Context Protocol	HIGH — primary announcement	https://www.anthropic.com/news/model-context-protocol
8	DeepEval — LLM-as-a-Judge Guide	HIGH — open-source eval framework documentation, actively maintained	https://deepeval.com/guides/guides-llm-as-a-judge
9	DeepEval — Unit Testing in CI/CD	HIGH — primary documentation	https://deepeval.com/docs/evaluation-unit-testing-in-ci-cd

10	DeepEval — RAG Evaluation Guide	HIGH — primary documentation	https://deepeval.com/guides/guides-rag-evaluation
11	RAGAS — Synthetic Testset Generation	HIGH — primary documentation	https://docs.ragas.io/en/v0.2.1/getstarted/rag_testset_generation/
12	Arize Phoenix — What is Arize Phoenix?	HIGH — primary documentation, open-source observability platform	https://arize.com/docs/phoenix
13	Langfuse — MCP Agent Evaluation (Pydantic AI Cookbook)	HIGH — practitioner cookbook, primary source	https://langfuse.com/guides/cookbook/example_pydantic_ai_mcp_agent_evaluation
14	Sundeep Teki — 4 Portfolio Projects Every AI FDE Should Build in 2026	MEDIUM — practitioner perspective, named AI career advisor	https://www.sundeepteki.org/advice/the-4-portfolio-projects-every-ai-forward-deployed-engineer-should-build-in-2026
15	AgenticCareers — AI Agent Portfolio Projects That Get You Hired in 2026	MEDIUM — practitioner-aggregated hiring signals	https://agenticcareers.co/blog/ai-agent-portfolio-projects-get-hired-2026
16	Digital Applied — AI Developer Hiring 2026: Skills That Actually Matter	MEDIUM — industry hiring analysis	https://www.digitalapplied.com/blog/ai-developer-hiring-skills-that-matter-2026
17	Digital Applied — Hybrid Search: BM25, Vector & Reranking 2026	MEDIUM — technical reference article	https://www.digitalapplied.com/blog/hybrid-search-bm25-vector-reranking-reference-2026
18	Digital Applied — Prompt Caching 2026	MEDIUM — engineering guide with vendor pricing	https://www.digitalapplied.com/blog/prompt-caching-2026-cut-llm-costs-engineering-guide
19	TowardsAI — Production RAG: Chunking, Retrieval, and Evaluation Strategies	MEDIUM — practitioner write-up with measured results	https://towardsai.net/p/machine-learning/production-rag-the-chunking-retrieval-and-evaluation-strategies-that-actually-work
20	Getmaxim — Prompt Injection Defense for		

	Production AI Agents 2026	MEDIUM — security-focused practitioner guide	https://www.getmaxim.ai/articles/prompt-injection-defense-for-production-ai-agents-a-complete-2026-guide/
21	Zenvanriel — Six-Figure AI Engineering Portfolio	MEDIUM — practitioner case study, specific metrics cited	https://zenvanriel.com/ai-engineer-blog/100k-ai-engineering-portfolio-projects/
22	Learnist — 12 Red Flags in AI Case Studies (2026)	MEDIUM — aggregated hiring practitioner perspective	https://www.learnist.org/ai-case-study-red-flags-portfolio-2026/
23	Abbacus Technologies — How to Hire an AI Developer 2026: Portfolio Red Flags	MEDIUM — hiring-manager perspective	https://www.abbacustechnologies.com/how-to-hire-an-ai-developer-in-2026-portfolio-red-flags-and-technical-must-haves/
24	BrainTrust — Best AI Eval Tools for CI/CD Pipelines 2026	MEDIUM — evaluation platform comparison	https://www.braintrust.dev/articles/best-ai-evals-tools-cicd-2025
25	BentoML — Key Metrics for LLM Inference (LLM Inference Handbook)	HIGH — framework vendor, primary technical reference	https://bentoml.com/llm/inference-optimization/llm-inference-metrics
26	arxiv 2411.13768 — EDDOps: Evaluation-Driven Development and Operations of LLM Agents	HIGH — peer-reviewed reference architecture	https://arxiv.org/html/2411.13768v3
27	StackAI — RAG Best Practices: Chunking, Embeddings, Reranking, Hybrid Search	MEDIUM — enterprise practitioner guide	https://www.stackai.com/insights/retrieval-augmented-generation-(rag)-best-practices-for-enterprise-ai-chunking-embeddings-reranking-and-hybrid-search-optimization
28	Braintrust — What Is LLM Monitoring?	MEDIUM — observability platform, technical reference	https://www.braintrust.dev/articles/what-is-llm-monitoring
29	Confident AI / DeepEval Blog — How to Evaluate RAG in CI/CD	HIGH — primary tooling documentation	https://www.confident-ai.com/blog/how-to-evaluate-rag-applications-in-ci-cd-pipelines-with-deepeval
30	RAGAS Paper — Automated Evaluation of Retrieval Augmented Generation	HIGH — original research paper	https://arxiv.org/abs/2309.15217